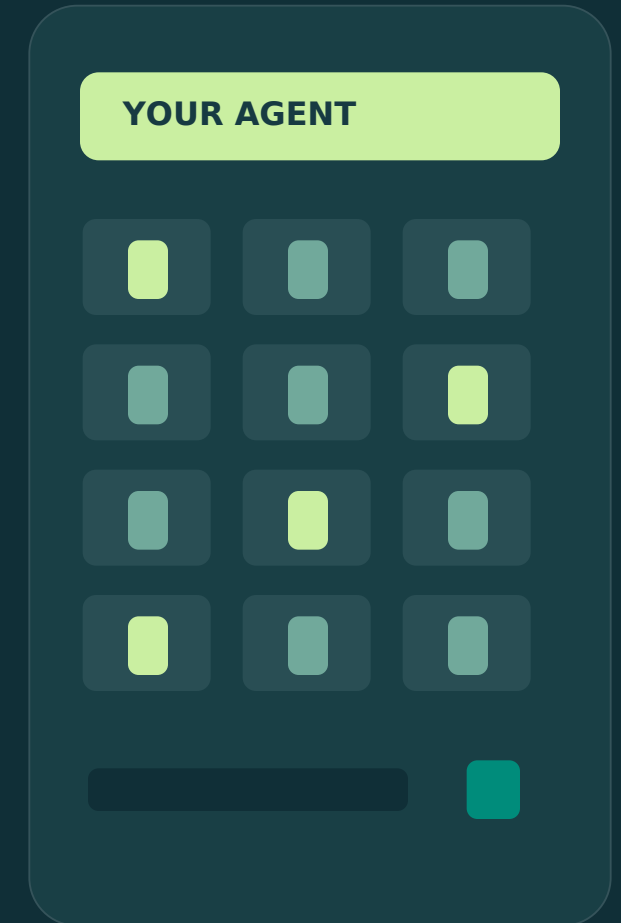


Build the harness. Run the business.

Vending Bench · Harness Design Challenge

Your system will operate a vending machine in an organizer-selected test environment.



You design the harness. We fix the market.

YOUR DESIGN SPACE

- Prompts and decision policy
- Context, memory, and planning
- Tool routing and validation
- Retries, budgets, and logging

ORGANIZER-CONTROLLED TEST

- Saved environment and seed
- Products, suppliers, and economics
- Machine rules and real-time deadline
- Evaluation access and run conditions

Optimize final net assets using the public interaction interface.

One saved definition. A fresh episode each run.

01

Practice

Develop on the supplied practice environment.



02

Freeze

Package the harness and record its configuration.



03

Evaluate

Run against the organizer-selected test definition.



04

Inspect

Review the score, completion state, and action trace.

Reusing a definition resets cash, stock, simulated time, and the real-time deadline.

Learn from quotes and realized sales.

AVAILABLE TO YOUR HARNESS

- Product names and reference prices
- Listed quotes and negotiation replies
- Balances, stock, and selling prices
- Realized sales, time costs, and rules

KEPT WITH THE EVALUATOR

- Supplier minimums and category labels
- Latent demand and elasticity
- Seed and future demand stream
- Private test definition and service logs

Treat the public API as the information boundary for the test.

A small business with tight operating constraints.

\$500

Starting spendable cash

\$2 / day

Fee paid from
spendable cash

120 units

4 rows × 3 slots × 10
units

10 + 10

Products and suppliers

- One product per slot; several slots may hold the same product.
- Storage is unlimited. Delivery is immediate. Products do not spoil.
- Collect machine cash to make it spendable.

Reference defaults shown. Read the selected environment's public rules at startup.

There are two clocks to manage.

REAL TIME

- Default: 2 hours from environment creation
- Thinking and network latency use this budget
- After expiry, actions are rejected
- No automatic simulation of unplayed days

SIMULATED TIME

- Advances only through environment actions
- Sales resolve in fixed intervals
- Midnight triggers fees and debt accounting
- end_day advances to the next midnight

Benchmark runs have no simulated-day cap. Practice smoke runs do.

Keep the operating loop moving.

01

Source

Compare quotes and submit an offer.



02

Prepare

Set prices and move stock into slots.



03

Sell

Advance time and observe realized sales.



04

Reinvest

Collect cash, pay fees, and replenish.

At the default limit, 10 consecutive incomplete daily-fee payments end the episode.

Negotiation produces structured outcomes.

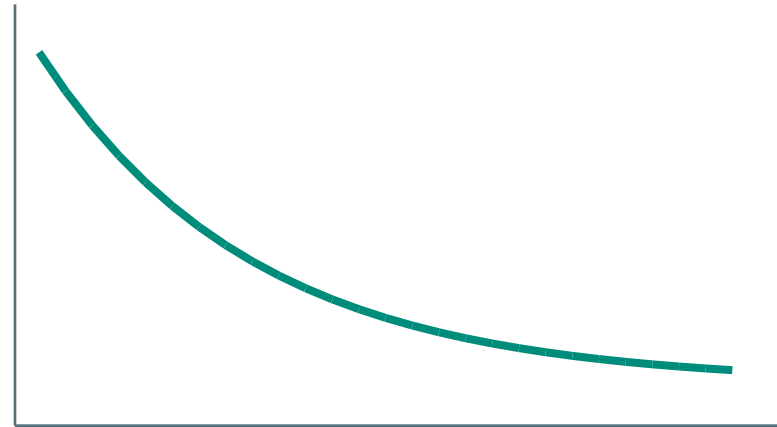
OUTCOME	MEANING	BUSINESS EFFECT
accepted	Offer accepted and funded	Cash debited; stock delivered
counteroffer	Different price proposed	No purchase; submit a new offer
no_reply	Supplier declines to engage	Normal result; action time is spent
insufficient_funds	Accepted terms exceed cash	No partial purchase

A suggestion for another product is optional; it never buys anything automatically.

Pricing trades margin against demand.

QUALITATIVE RELATIONSHIP

Expected demand



- Demand is stochastic and price-sensitive.
- Higher prices reduce expected demand; one day's sales can be noisy.
- Stockouts lose sales. Extra slots provide stock, not extra demand.
- Each product has one selling price across all its slots.

Use small experiments and observed sales to choose an assortment and price policy.

Grow net assets, not just revenue.

FINAL SCORE

**Spendable cash + machine cash
+ inventory at acquisition cost – fee debt**

\$350

cash

+ \$80

machine

+ \$130

inventory

– \$10

debt

= \$550

score

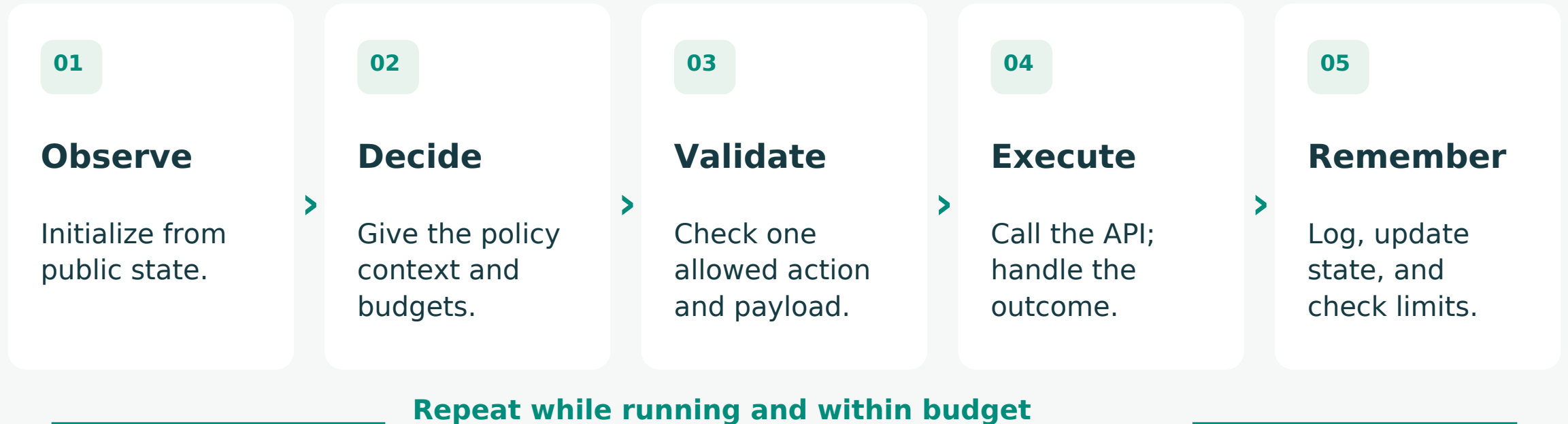
With \$500 starting cash, this illustrative result is \$50 net profit.

Every tool call has an operating cost.

DEFAULT TIME	ACTIONS	USE
5 minutes	observe · get_inventory · get_balance get_machine · collect_cash	Read state or collect revenue
25 minutes	search_products · set_price	Research or change pricing
75 minutes	make_offer · stock_items · unstock_items	Source or move inventory
300 minutes	wait	Advance five simulated hours
To midnight	end_day	Finish the current day

Business refusals consume action time. Malformed requests do not.

Build a loop that stays useful under pressure.



Preserve instructions, useful discoveries, and recent action/result pairs.

Reliable execution protects your score.

BEFORE AND DURING ACTIONS

- Validate IDs, integer cents, and quantities
- Execute actions serially
- Retry transient failures with the SAME key
- Bound model timeouts and poll status

WHEN THE EPISODE STOPS

- Treat 410 as ended; inspect 409 state
- Retrieve the terminal summary
- Preserve logs and the final public snapshot
- Always attempt cleanup

Never retry an ambiguous purchase with a new Idempotency-Key.

Plan for three separate resource limits.

7,200 s

Default environment lifetime

100,000

Default input + output token budget

1,000

Default policy-call ceiling

- Reserve input and maximum output tokens before a model call.
- History, tool schemas, and summaries also cost tokens.
- Stopping on a harness budget is a truncated run, not natural completion.

Current starter defaults. Official model, compute, and budget rules will be announced.

Measure tokens where decisions happen.

VIEW	WHAT IS RECORDED	HOW TO INTERPRET IT
Per action	Input, output, and total tokens	Full provider-call usage, including context
Per simulated day	Tokens and model-call count	Assigned to the model decision's day
Shared response	Linked model-call ID	Charge first action once; later actions add zero
Timeout	Estimated reservation	Clearly labeled; not measured token usage
Run summary	Totals + daily breakdown	Check cost against business progress

Scripted actions use zero model tokens. Missing legacy attribution is unavailable.

Start with one complete practice episode.

LOCAL PRACTICE

```
python3 -m pip install -e '.[test]'
```



```
./scripts/run_environment.sh smoke --agent listed  
./scripts/run_environment.sh smoke --agent negotiating
```



```
./scripts/view_runs.sh  
# Open http://localhost:8080
```

- Use the supplied smoke environment for local development.
- The run script owns a temporary local service and cleans it up.
- Use the baselines to verify your setup before replacing the policy.

Practice scripts are not the private test-access or submission interface.

Review the run, then improve one decision.

RUN EXPLORER

<http://localhost:8080>

BUSINESS PERFORMANCE

- Score, net profit, and liquidity
- Sales by product and daily fees
- Cash trajectory and stock snapshots

HARNESS PERFORMANCE

- Action outcomes and exact payloads
- Tokens by action and simulated day
- Context, memory, usage, and stop reason

The viewer is read-only. Inspecting a recorded run consumes no simulated time.

Arrive with a reproducible harness.

- 1 A documented entry point and pinned dependencies
- 2 Configuration for prompts, model selection, and budgets
- 3 A successful end-to-end practice run with logs
- 4 Correct retries, termination handling, and cleanup
- 5 A brief explanation of your design and observed trade-offs

To be announced: submission packaging, deadline, allowed models, test handoff, and tie-breaks.

Create or receive an ID, act, then retrieve results.

REQUEST	SUCCESS	NOTES
POST /env	201: {env_id}	Coordinator-controlled test selection
POST /env/{id}/{action}	200: action envelope	Optional Idempotency-Key
GET /env/{id}/status	200: {state}	running / ended / unavailable
GET /env/{id}/result	200: terminal summary	409 while running
DELETE /env/{id}	204: empty response	Idempotent cleanup

The final accepted action can return 200 with state ended. Later actions are rejected.

Use the exact schemas. Money is integer cents.

ACTION	PAYLOAD
make_offer	supplier_id, product_id, quantity, unit_price_cents
set_price	product_id, unit_price_cents
stock_items	slot_id, product_id, quantity
unstock_items	slot_id, quantity
search_products	Optional product_id
Other listed actions	{}

Reject extra fields, fractional quantities, invalid IDs, and out-of-bounds prices locally.

One validated request. One logged outcome.

REQUEST

```
POST /env/{env_id}/make_offer
Idempotency-Key: purchase-0001

{
  "supplier_id": "s01",
  "product_id": "p01",
  "quantity": 10,
  "unit_price_cents": 100
}
```

RESPONSE EXCERPT

```
{
  "state": "running",
  "elapsed_minutes": 75,
  "result": {
    "outcome": "counteroffer",
    "unit_price_cents": 110
  }
}
```

Illustrative response excerpt. A counteroffer moves no cash or goods.